Chapter 4 - VideoChip

VideoChip is the IE's general-purpose framebuffer chip. It sits at the bottom of the compositor stack (layer `0`) and provides three things: a framebuffer in main memory, a programmable raster engine called the **copper list**, and a 2D drawing accelerator called the **blitter**. There is no BASIC keyword that addresses VideoChip directly; the chip is driven by `POKE` into its register block from BASIC, or by ordinary stores from any of the six CPUs.

4.1 Where the picture lives

VideoChip reads its picture from a region of main memory called **VRAM**. VRAM is a five-megabyte region that begins at byte address `0x00100000` (`1` MB) and ends at `0x005FFFFF`. Any byte in that region is visible to the chip on the next frame.

```
+--------------------------+
|  VRAM                    |
|  0x00100000 .. 0x005FFFFF |
|  5 MB                    |
+--------------------------+
```

Five megabytes is enough to hold several frames at the smaller modes (a `960` × `540` RGBA frame is `2,073,600` bytes, so two such frames fit with room to spare), but **not** enough for one full `1920` × `1080` RGBA frame, which requires `8,294,400` bytes. In the `1920` × `1080` mode, point `FB_BASE` at a region of ordinary main memory large enough for one frame, instead of into VRAM.

The chip does not own VRAM exclusively. Any CPU can read and write the same bytes. When the compositor builds the next frame, it picks up whatever was last written.

4.2 The register block

Every VideoChip register is a 32-bit word at a fixed address. The block begins at `0x000F0000` and ends at `0x000F049B`. All registers are aligned to 4 bytes.

Address	Name	Purpose
`0xF0000`	`VIDEO_CTRL`	Master enable. `0` = off, non-zero = on.
`0xF0004`	`VIDEO_MODE`	Output mode (table below).
`0xF0008`	`VIDEO_STATUS`	Status bits (read-only).
`0xF000C`	`COPPER_CTRL`	Copper enable and reset.
`0xF0010`	`COPPER_PTR`	Base address of copper program in main memory.
`0xF0014`	`COPPER_PC`	Copper program counter (read-only).
`0xF0018`	`COPPER_STATUS`	Copper status bits (read-only).
`0xF001C`	`BLT_CTRL`	Blitter start and IRQ enable.
`0xF0020`	`BLT_OP`	Blitter operation.
`0xF0024`	`BLT_SRC`	Blitter source address.
`0xF0028`	`BLT_DST`	Blitter destination address.

Address	Name	Purpose
0xF002C	BLT_WIDTH	Blitter rectangle width, in pixels (or bytes for CLUT8).
0xF0030	BLT_HEIGHT	Blitter rectangle height, in pixels.
0xF0034	BLT_SRC_STRIDE	Source stride in bytes per row.
0xF0038	BLT_DST_STRIDE	Destination stride in bytes per row.
0xF003C	BLT_COLOR	Fill colour or line colour.
0xF0040	BLT_MASK	Per-pixel mask for masked copies.
0xF0044	BLT_STATUS	Blitter status bits (read-only).
0xF0048	RASTER_Y	Scanline at which raster IRQ fires.
0xF004C	RASTER_HEIGHT	Raster band height.
0xF0050	RASTER_COLOR	Raster band fill colour.
0xF0054	RASTER_CTRL	Raster band start (write 1).
0xF0058	BLT_MODE7_U0	Mode 7 texture origin (U).
0xF005C	BLT_MODE7_V0	Mode 7 texture origin (V).
0xF0060	BLT_MODE7_DU_COL	Mode 7 U step per column.
0xF0064	BLT_MODE7_DV_COL	Mode 7 V step per column.
0xF0068	BLT_MODE7_DU_ROW	Mode 7 U step per row.
0xF006C	BLT_MODE7_DV_ROW	Mode 7 V step per row.
0xF0070	BLT_MODE7_TEX_W	Mode 7 texture width.
0xF0074	BLT_MODE7_TEX_H	Mode 7 texture height.
0xF0078	PAL_INDEX	Auto-incrementing palette write index (0–255).
0xF007C	PAL_DATA	Palette data port. Writes entry, advances PAL_INDEX.
0xF0080	COLOR_MODE	0 = RGBA32 direct, non-zero = CLUT8 indexed.
0xF0084	FB_BASE	Byte address in main memory where the framebuffer begins.
0xF0088–0xF0487	PAL_TABLE	Direct palette window: 256 entries of 4 bytes each.
0xF0488	BLT_FLAGS	Blitter BPP and draw-mode flags.
0xF048C	BLT_FG	Blitter foreground colour for colour-expand.
0xF0490	BLT_BG	Blitter background colour for colour-expand.
0xF0494	BLT_MASK_MOD	Mask-row stride for colour-expand.
0xF0498	BLT_MASK_SRCX	Mask source X offset.

Reads of write-only fields and writes to read-only fields are silently ignored.

4.3 Enabling the chip and choosing a mode

VideoChip starts disabled. To turn it on, write a non-zero value to VIDEO_CTRL. To turn it off, write 0.

The chip supports eight output modes. Each mode selects a frame width and height. The default mode at power-on is `0x07` (`960 × 540`).

Value	Mode
`0x00`	`640 × 480`
`0x01`	`800 × 600`
`0x02`	`1024 × 768`
`0x03`	`1280 × 960`
`0x04`	`320 × 200`
`0x05`	`320 × 240`
`0x06`	`1920 × 1080`
`0x07`	`960 × 540`

Write the desired value into `VIDEO_MODE`. The chip resizes its internal framebuffer and notifies the compositor. Writing a value outside the table is ignored.

Below is a BASIC fragment that turns the chip on at `960 × 540` and sets the framebuffer base at the start of VRAM:

```
10 POKE &H000F0004, &H07              : REM VIDEO_MODE = 960x540
20 POKE &H000F0084, &H00100000        : REM FB_BASE   = VRAM start
30 POKE &H000F0000, 1                 : REM VIDEO_CTRL = on
```

`POKE` writes the four bytes of a 32-bit word at the given aligned address (see Chapter 2).

4.4 The framebuffer

`FB_BASE` is the byte address in main memory where the chip reads the first pixel of the picture. The chip reads `W × H` pixels in left-to-right, top-to-bottom order, where `W` and `H` are the width and height of the current mode.

There are two pixel formats, selected by `COLOR_MODE`:

`COLOR_MODE`	Pixel format	Bytes per pixel
`0`	RGBA32 direct colour	4
non-zero	CLUT8 indexed colour	1

In **RGBA32** mode every pixel is four bytes, little-endian, in the order red, green, blue, alpha. The compositor uses the alpha byte for mask blending as described in Chapter 3.

In **CLUT8** mode every pixel is one byte, which is an index into the 256-entry CLUT palette. The chip expands each index into an RGBA32 pixel on the fly. The alpha byte produced by the expansion is always `0xFF` (fully opaque).

`FB_BASE` may be set anywhere in main memory. There is no requirement to point it at the VRAM window; any address that holds `W × H × bytes-per-pixel` bytes of valid data works. Pointing it into VRAM means writes through ordinary memory stores update the picture immediately.

4.5 The palette

The palette has 256 entries. Each entry is a 24-bit RGB triple packed in the low three bytes of a 32-bit word:

```
bit 31           24 23           16 15            8 7             0
+---------------+---------------+---------------+---------------+
|   reserved    |     red       |    green      |     blue      |
+---------------+---------------+---------------+---------------+
```

There are two ways to set palette entries.

Indexed port. Write the entry number (`0`–`255`) to `PAL_INDEX`. Then write each entry's 24-bit value to `PAL_DATA`. The chip stores the value, increments `PAL_INDEX`, and wraps at 256. This is the shortest way to load a whole palette:

```
100 POKE &H000F0078, 0          : REM PAL_INDEX = 0
110 POKE &H000F007C, &HFF0000   : REM entry 0 = red
120 POKE &H000F007C, &H00FF00   : REM entry 1 = green
130 POKE &H000F007C, &H0000FF   : REM entry 2 = blue
```

Direct window. Each entry is also visible at `PAL_TABLE + index*4`. Writing the word at that address updates entry `index` and leaves `PAL_INDEX` unchanged. Use this when you want to set a single entry without disturbing a write cursor.

The palette is shared between CLUT8 picture display and the blitter's colour-expand operation.

4.6 The blitter

The blitter is a 2D copy and fill engine. It reads from a source rectangle in main memory, performs a per-pixel operation, and writes the result to a destination rectangle.

4.6.1 Operations

`BLT_OP` selects one of eight operations:

BLT_OP	Operation	What it does
0	COPY	Copy source to destination.
1	FILL	Fill destination with BLT_COLOR.
2	LINE	Draw a line from corner to corner in BLT_COLOR.
3	MASKED_COPY	Copy where BLT_MASK selects source, else leave destination.
4	ALPHA_COPY	Copy only pixels whose alpha byte is non-zero.
5	MODE7	Affine texture mapping with the Mode-7 registers.
6	COLOR_EXPAND	Expand a 1-bit-per-pixel mask into BLT_FG/BLT_BG pixels.
7	SCALE	Nearest-neighbour scale source to destination size.

4.6.2 Setting up a transfer

The general procedure is:

1. Write the source and destination addresses to `BLT_SRC` and `BLT_DST`.
2. Write the rectangle size to `BLT_WIDTH` and `BLT_HEIGHT`.
3. Write the row strides in bytes to `BLT_SRC_STRIDE` and `BLT_DST_STRIDE`.
4. Write any operation-specific registers (`BLT_COLOR`, `BLT_MASK`, `BLT_FG`, `BLT_BG`, the Mode-7 set, etc.).
5. Write the operation number to `BLT_OP`.

6. Write the pixel format and draw mode to `BLT_FLAGS`.
7. Write `1` to `BLT_CTRL` to start.

4.6.3 `BLT_FLAGS` bits

Bits	Field	Meaning
1–0	BPP	`0` = RGBA32 (4 bytes per pixel), `1` = CLUT8 (1 byte per pixel).
7–4	Draw mode	One of 16 raster operations applied per pixel.
8	JAM1	Skip background pixels in colour-expand.
9	Invert template	Invert template bits before processing.
10	Invert mode	XOR destination with all-ones for set template bits.

4.6.4 Status and completion

`BLT_STATUS` carries three read-only bits:

Bit	Name	Meaning
0	`ERR`	The last transfer failed (out-of-range address).
1	`DONE`	The last transfer completed.
2	`IRQ_PENDING`	Blitter IRQ is pending.

The blitter runs synchronously: when the write to `BLT_CTRL` returns, the operation has finished and `DONE` is set. Setting bit `2` of `BLT_CTRL` enables blitter IRQs; the CPU then receives an interrupt instead of polling. To clear a pending IRQ, write `1` to bit `2` of `BLT_STATUS`.

4.7 The copper list

The copper is a tiny processor that runs through the rendering of a frame and writes registers at chosen scanlines. Its program lives in main memory; the chip reads it word by word.

To start the copper, write the address of the first word to `COPPER_PTR`, then write `1` to bit `0` of `COPPER_CTRL`. The copper resets at the start of every frame. To halt and reset explicitly, write bit `1` of `COPPER_CTRL`.

4.7.1 Instruction words

Each instruction is a 32-bit word. The top two bits select the opcode:

Top bits	Opcode	Length	Effect
`00`	`WAIT`	1 word	Wait until the raster reaches the given (Y, X).
`01`	`MOVE`	2 words	Write the second word to the register named by the first.
`10`	`SETBASE`	1 word	Change the base address used by `MOVE`.
`11`	`END`	1 word	Halt the copper for the rest of this frame.

WAIT packs `Y` in bits `23–12` and `X` in bits `11–0`. The copper resumes when the raster's Y exceeds the given Y, or when the raster's Y equals the given Y and X is at least the given X.

MOVE packs the register index in bits 23–16 of the first word. The address written is `IO_BASE + (index * 4)`. The second word holds the data.

SETBASE changes `IO_BASE`. The new base is stored in bits 23–0 of the instruction, shifted left by 2 (so any 4-byte aligned address up to 26 bits is reachable). At the start of every frame, `IO_BASE` is reset to `0x000F0000` (the VideoChip register block).

END halts the copper for the remainder of the current frame. The copper restarts from `COPPER_PTR` at the next frame.

4.7.2 A short example

This copper program changes `BLT_COLOR` to red at scanline `100` and to blue at scanline `200`, then halts:

```
; opcode  Y     X     word
; WAIT 100,0                  -> 0x00064000
; MOVE BLT_COLOR (idx 0x0F)   -> 0x400F0000, 0x00FF0000
; WAIT 200,0                  -> 0x000C8000
; MOVE BLT_COLOR              -> 0x400F0000, 0x000000FF
; END                         -> 0xC0000000
```

Register index `0x0F` is `BLT_COLOR` (`0xF003C` minus `0xF0000`, divided by `4`).

4.7.3 Copper status

`COPPER_STATUS` has three read-only bits:

Bit	Name	Meaning
0	`RUNNING`	The copper is enabled and not halted.
1	`WAITING`	The copper is paused on a `WAIT` instruction.
2	`HALTED`	The copper has hit `END` (or a bad opcode) this frame.

`COPPER_PC` reads back the address of the next instruction.

4.8 The raster band

The raster band is a single-shot fill of one or more scanlines. Write the starting Y to `RASTER_Y`, the number of scanlines to `RASTER_HEIGHT`, and the RGBA colour to `RASTER_COLOR`. Then write `1` to bit `0` of `RASTER_CTRL`. The chip fills the band on the current frame and ignores the request on later frames until the start bit is written again.

The raster band is the simplest way to draw a horizontal stripe without programming the blitter.

4.9 `VIDEO_STATUS`

`VIDEO_STATUS` is read-only and reports three bits:

Bit	Name	Meaning
0	`HAS_CONTENT`	At least one pixel has been written to VRAM since the chip was enabled.
1	`VBLANK`	The chip is currently in the vertical-retrace interval.
2	`FB_ERR`	The framebuffer source is unreachable or undersized for the current mode.

The `VBLANK` bit is the safest place to poll if you have no interrupt handler installed and need to wait for the retrace.

4.10 Putting it together

A minimal "set a mode, load a palette, draw a frame" sequence from machine language looks like this:

1. Write `VIDEO_MODE` and `FB_BASE`.
2. Write `COLOR_MODE` (`0` for RGBA32, non-zero for CLUT8).
3. If CLUT8: write 256 palette entries via `PAL_INDEX`/`PAL_DATA`.
4. Write `VIDEO_CTRL` = `1`.
5. Fill the framebuffer with whatever you want to display.
6. Optionally start the copper to change registers per-scanline, start the blitter for fast copies, or program a raster band.

Chapter 23 describes how the address constants above appear in machine-language source. Chapter 10 shows how tile and sprite layers map onto the framebuffer.